

Hospital Readmission Prediction Using Machine Learning

An Interactive Healthcare Analytics Dashboard Based on the UCI Diabetes 130-US
Hospitals Dataset

Dr. Neha Malav

Healthcare Analytics • Machine Learning • Artificial Intelligence

Final revised preprint

This version reports a reproducible experiment using the supplied dataset and the supplied Streamlit implementation. Numerical results are recomputed from those materials rather than copied from placeholder values in the earlier project report.

2026

Abstract

Hospital readmission is an important healthcare quality and resource-utilization outcome. This study presents an educational and reproducible machine-learning workflow for binary hospital-readmission prediction using the UCI Diabetes 130-US Hospitals dataset. The supplied dataset contains 101,766 hospital encounters and 50 variables. The implemented workflow replaces placeholder missing values, removes encounter and patient identifiers, imputes remaining missing values, converts the original three-level readmission outcome into a binary target, label-encodes categorical variables, and trains a Random Forest classifier with 100 trees. Using the supplied implementation settings—an 80/20 train-test split and `random_state=42`—the reproduced model achieved 63.50% accuracy on the held-out test set. For the positive class, precision was 62.61%, recall was 52.08%, and F1-score was 56.87%. The dashboard combines dataset inspection, exploratory data analysis, model evaluation, feature-importance visualization, and an interactive risk-estimation interface. An important implementation distinction is documented: the dashboard's interactive Prediction module uses a manually constructed heuristic score rather than the trained Random Forest probability. It is therefore an illustrative interface, not a calibrated clinical risk model. The study should be interpreted as a software and analytics prototype. It does not establish clinical validity, prospective utility, fairness, or safety.

Keywords: hospital readmission; diabetes; machine learning; Random Forest; healthcare analytics; Streamlit; predictive modeling; clinical decision support; explainability

Research contribution

The principal contribution is an end-to-end prototype that connects a public healthcare dataset, preprocessing, exploratory analysis, supervised learning, model evaluation, global feature importance, and an interactive web interface. The work also provides a transparent account of implementation limitations so that the prototype is not presented as a clinically validated decision-support system.

Table of Contents

Section	Title
1	Introduction
2	Related Work and Conceptual Background
3	Dataset and Problem Formulation
4	Data Preprocessing
5	Exploratory Data Analysis
6	Machine-Learning Methodology
7	Interactive Dashboard Implementation
8	Experimental Results
9	Interpretation and Discussion
10	Limitations, Risks, and Responsible Use
11	Reproducibility and Deployment
12	Future Work
13	Conclusion
	References
A	Appendix: Dataset Variable Inventory
B	Appendix: Reproduced Classification Report
C	Appendix: Key Implementation Details

1. Introduction

Hospital readmission is widely used as an indicator of healthcare utilization and quality. Repeated admissions can increase resource consumption and may identify patients who require more effective transitions of care, medication management, follow-up, or chronic-disease support. The increasing availability of structured electronic health-record data creates opportunities to study readmission patterns using statistical and machine-learning methods.

This project focuses on the Diabetes 130-US Hospitals dataset and demonstrates a complete analytical pipeline from raw tabular data to an interactive application. The project is intentionally positioned as an educational and portfolio prototype. The objective is not to replace clinicians or claim that a model is ready for clinical deployment.

1.1 Problem statement

The central problem is whether structured clinical and healthcare-utilization variables can be transformed into a supervised-learning representation that distinguishes encounters associated with subsequent readmission from encounters without readmission, while also presenting the analytical workflow through an accessible dashboard.

1.2 Objectives

- Inspect and characterize the supplied hospital-encounter dataset.
- Clean missing and placeholder values and remove direct identifier fields.
- Transform the original readmission outcome into a binary prediction target.
- Train a Random Forest classifier under the settings implemented in the supplied application.
- Evaluate performance using accuracy, precision, recall, F1-score, and a confusion matrix.
- Inspect global Random Forest feature importance.
- Provide an interactive Streamlit interface for data exploration and illustrative risk estimation.
- Document methodological limitations and responsible-use constraints.

1.3 Scope

The scope is limited to the supplied dataset, the supplied application logic, and a single reproducible train-test split. No external validation cohort, prospective evaluation, calibration study, subgroup fairness analysis, hyperparameter search, or clinical workflow study is included.

2. Related Work and Conceptual Background

Earlier research using the diabetes hospital dataset examined associations between clinical variables, HbA1c measurement, medication changes, utilization, and readmission outcomes. The supplied project report also reviews traditional statistical approaches, machine learning, Random Forest, explainable AI, and deep learning.

2.1 Random Forest

Random Forest is an ensemble method that constructs multiple decision trees using randomized sampling and feature selection, then aggregates their predictions. It is useful for structured tabular data because it can represent nonlinear relationships and interactions without requiring a single linear functional form.

2.2 Explainability

Healthcare prediction requires particular care with interpretation. Global feature importance can indicate which variables contribute most to the fitted forest, but it is not equivalent to causal influence and does not explain an individual prediction. Methods such as SHAP and LIME are established explanation approaches, but they are not implemented in the supplied application. The current prototype uses Random Forest feature importance as a basic global interpretability mechanism.

2.3 Position of this study

The distinctive emphasis of the project is integration rather than algorithmic novelty. Dataset exploration, preprocessing, model training, evaluation, feature importance, and an interactive dashboard are brought together in one application. The scientific value of the current version therefore lies primarily in reproducible implementation and transparent documentation of what the system does and does not establish.

2.4 Research gap addressed

The project addresses a practical gap between a standalone machine-learning notebook and an accessible analytical application. However, the current implementation should not be described as solving the broader clinical problem of readmission prediction because external validation and prospective evaluation are absent.

3. Dataset and Problem Formulation

3.1 Dataset

The supplied project uses the Diabetes 130-US Hospitals dataset, originally collected from 130 U.S. hospitals over approximately ten years. The working CSV supplied with the project contains 101,766 rows and 50 columns. The original outcome variable *readmitted* has three categories: NO, >30, and <30.

Property	Value
Encounters	101,766
Variables	50
Hospitals represented	130
Original target	NO / >30 / <30
Binary negative class	54,864 (53.9%)
Binary positive class	46,902 (46.1%)
Model split	80% training / 20% testing
Random Forest trees	100
Random state	42

3.2 Target construction

The supplied implementation maps NO to 0 and both >30 and <30 to 1. This converts the original three-category outcome into a binary target representing any recorded readmission. The transformation simplifies the classification task but discards the distinction between early readmission (<30 days) and later readmission (>30 days).

3.3 Class balance

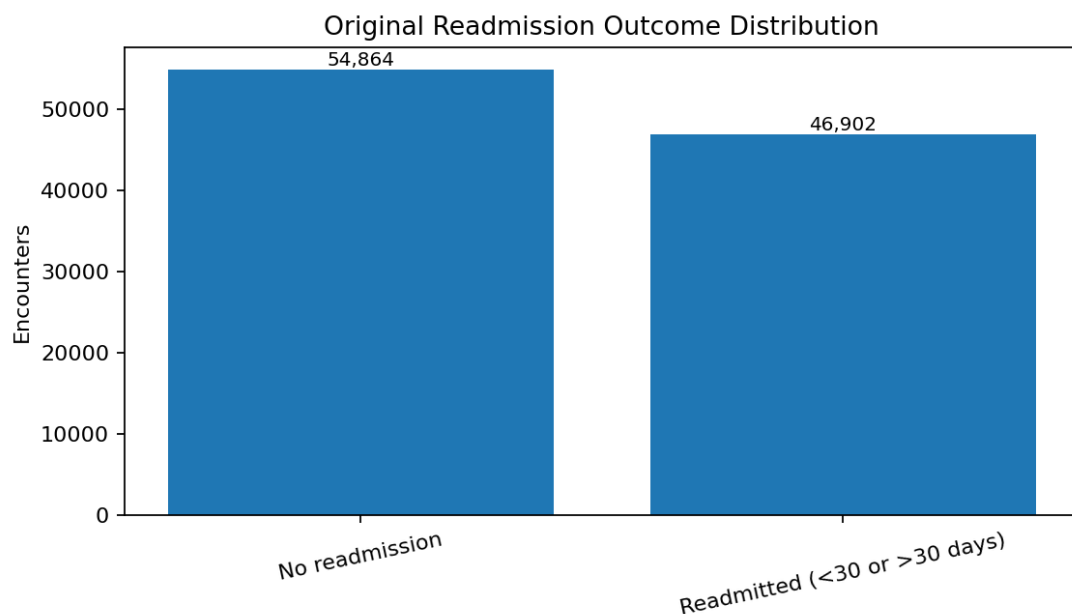


Figure 1. Original readmission outcome distribution. The binary positive class combines the two readmission categories used by the supplied implementation.

4. Data Preprocessing

4.1 Placeholder and missing-value handling

The application first replaces the literal '?' placeholder with missing values. Numeric variables are then imputed with their column median. Categorical variables are imputed with the mode when a mode exists; otherwise the value 'Unknown' is used.

4.2 Identifier removal

The fields encounter_id and patient_nbr are removed before modeling. This is appropriate for the prototype because they are identifiers rather than clinically meaningful predictive variables.

4.3 Categorical encoding

Remaining object-type variables are independently transformed with LabelEncoder. This creates integer representations that can be consumed by the Random Forest implementation. For a research-grade pipeline, a fitted preprocessing object should be learned only from the training data and applied unchanged to the test data and future observations. The current application instead performs preprocessing before the train-test split, which creates a potential information-leakage pathway for imputation and encoding.

4.4 Missingness profile

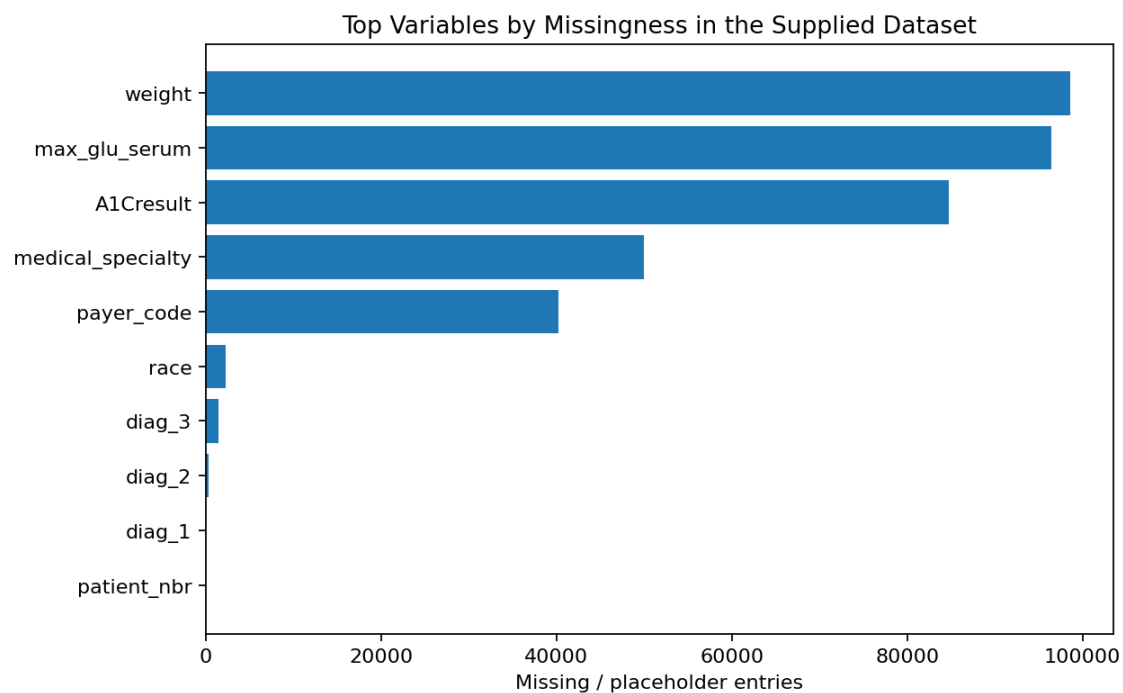


Figure 2. Ten variables with the highest counts of missing or placeholder entries in the supplied dataset.

Variable	Missing / placeholder	Approx. proportion
weight	98,569	96.9%
max_glu_serum	96,420	94.7%
A1Cresult	84,748	83.3%
medical_specialty	49,949	49.1%
payer_code	40,256	39.6%
race	2,273	2.2%
diag_3	1,423	1.4%

Variable	Missing / placeholder	Approx. proportion
diag_2	358	0.4%
diag_1	21	0.0%
patient_nbr	0	0.0%

5. Exploratory Data Analysis

The supplied dashboard includes visualizations for readmission distribution, gender, race, age, time in hospital, number of diagnoses, correlations, and missing values. The following figures reproduce key descriptive views directly from the supplied dataset.

5.1 Age distribution

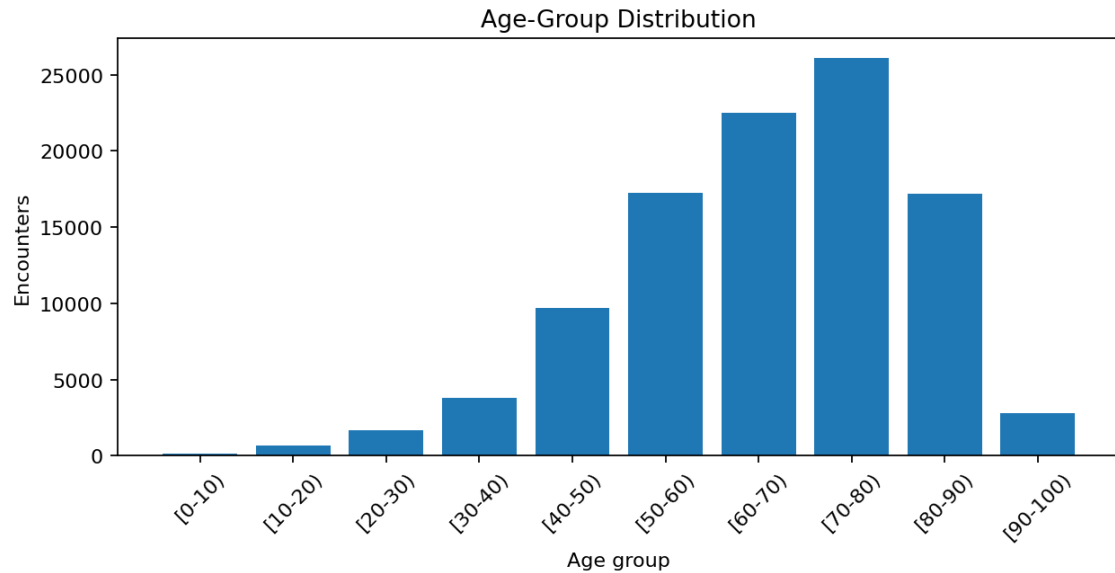


Figure 3. Distribution of encounters across age groups.

The dataset is concentrated in older age groups. The 70–80 age band is the largest represented group in the supplied data, followed by 60–70 and 80–90. This descriptive pattern should not be interpreted as a causal or population-level estimate because the dataset is a specific hospital-encounter collection.

5.2 Length of stay

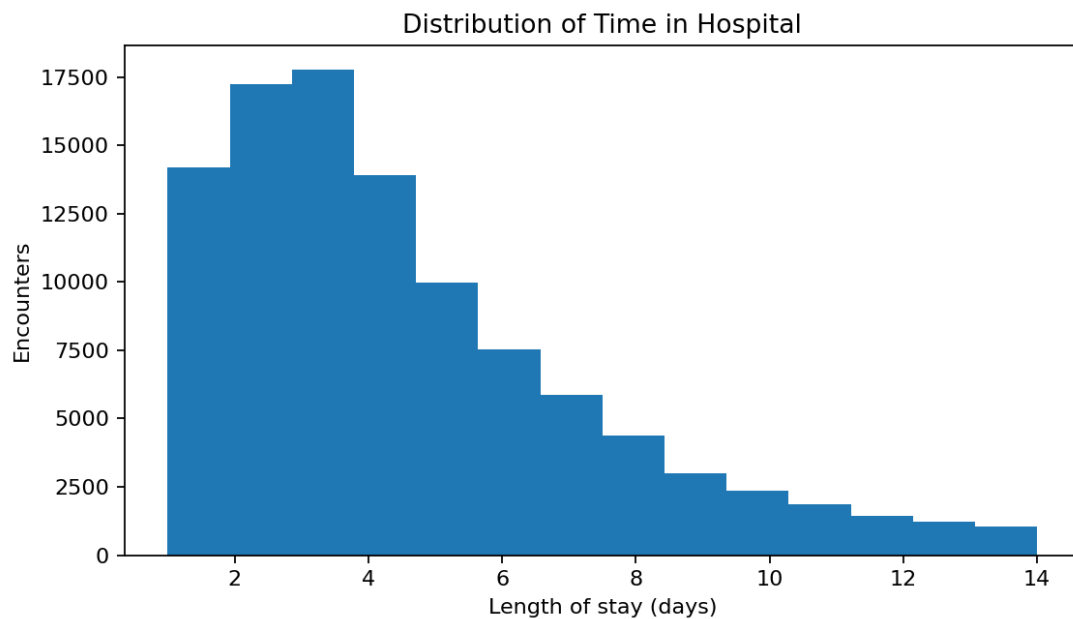


Figure 4. Distribution of time in hospital.

6. Machine-Learning Methodology

6.1 Model

The model is a RandomForestClassifier configured with `n_estimators=100` and `random_state=42`. The features are the processed columns after identifier removal and encoding; the target is the binary readmission variable.

6.2 Train-test split

The supplied implementation uses `train_test_split` with `test_size=0.20` and `random_state=42`. No `stratify` parameter is supplied. The resulting test set contains 20,354 encounters.

6.3 Evaluation metrics

Accuracy measures the proportion of correctly classified encounters. Precision for class 1 measures the proportion of predicted readmissions that are actually readmissions. Recall for class 1 measures the proportion of actual readmissions detected by the model. F1-score is the harmonic mean of precision and recall. Because the two classes are not perfectly balanced, class-specific metrics are more informative than accuracy alone.

6.4 Reproducible implementation

```
X = data.drop("readmitted", axis=1)
y = data["readmitted"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

model = RandomForestClassifier(
    n_estimators=100, random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

6.5 Interpretability output

The application ranks features using `model.feature_importances_`. These values are useful for global inspection of the fitted forest but should not be interpreted as causal effects, clinical importance, or individual-level explanations.

7. Interactive Dashboard Implementation

The Streamlit application is organized into six modules: Dashboard, Dataset, EDA, Machine Learning, Prediction, and About. The Dashboard module displays encounter counts, column counts, readmission counts, a data preview, dataset shape, and missing-value summaries. The EDA module provides descriptive plots. The Machine Learning module trains the Random Forest and displays evaluation outputs and feature importance.

7.1 Dashboard architecture

Module	Primary function
Dashboard	Dataset summary, preview, shape, missing-value overview
Dataset	Interactive inspection of the supplied data
EDA	Readmission, demographic, utilization, correlation, and missingness plots
Machine Learning	Train Random Forest; show accuracy, classification report, confusion matrix, feature importance
Prediction	Interactive illustrative risk score from manually entered variables
About	Project description, technologies, and developer information

7.2 Important distinction in the Prediction module

The supplied Prediction module does not call the trained Random Forest model. Instead, it calculates a manually constructed `risk_score` from age, time in hospital, medications, diagnoses, emergency visits, inpatient visits, outpatient visits, and laboratory procedures, then divides the score by 100 and caps it at 1.0. The resulting number is presented as an estimated readmission risk.

```
risk_score = (
    age * 0.15 +
    time_in_hospital * 4 +
    medications * 0.6 +
    diagnoses * 2 +
    emergency * 5 +
    inpatient * 5 +
    outpatient * 2 +
    num_lab * 0.05
)
probability = min(risk_score / 100, 1.0)
```

Interpretation: this is a heuristic demonstration, not a probability calibrated from the Random Forest and not a validated clinical risk score. The final paper explicitly preserves this distinction to avoid overstating the system's capability.

8. Experimental Results

The numerical results below were recomputed from the supplied CSV and the supplied Random Forest implementation using the same core settings documented in the project. This replaces the placeholder XX.XX values present in the earlier report.

8.1 Overall performance

Metric	Value
Accuracy	0.6350 (63.50%)
Precision — class 1	0.6261 (62.61%)
Recall — class 1	0.5208 (52.08%)
F1-score — class 1	0.5687 (56.87%)
Macro F1	0.6262 (62.62%)
Test observations	20,354

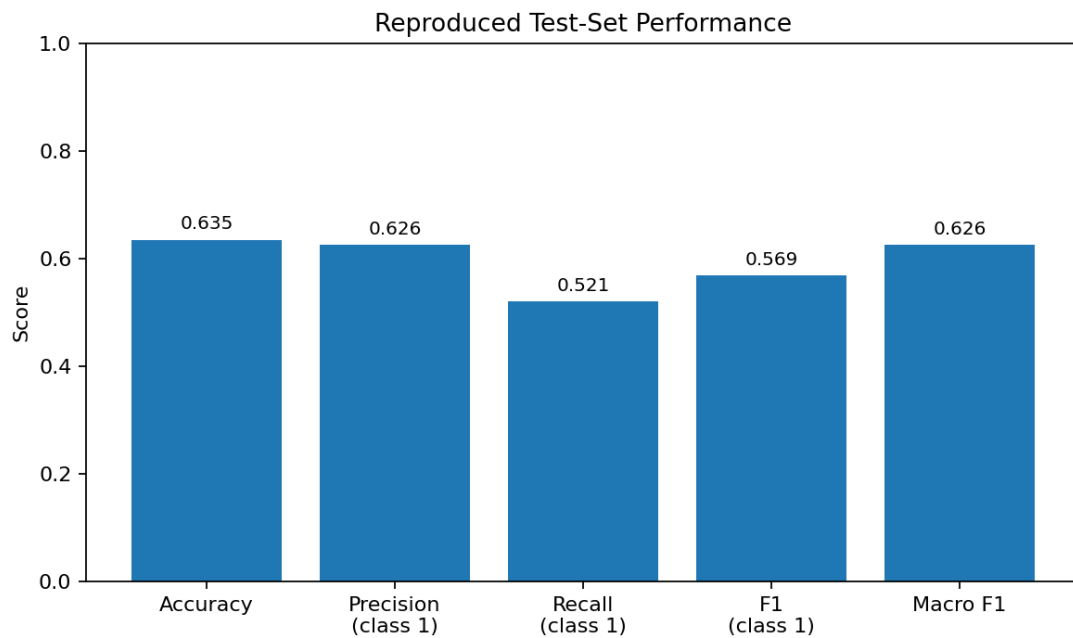


Figure 5. Reproduced test-set performance under the supplied model configuration.

8.2 Classification report

Class	Precision	Recall	F1	Support
0 — No readmission	0.6405	0.7330	0.6837	10,952
1 — Readmitted	0.6261	0.5208	0.5687	9,402
Macro average	0.6333	0.6269	0.6262	20,354
Weighted average	0.6339	0.6350	0.6305	20,354

9. Interpretation and Discussion

9.1 Confusion matrix

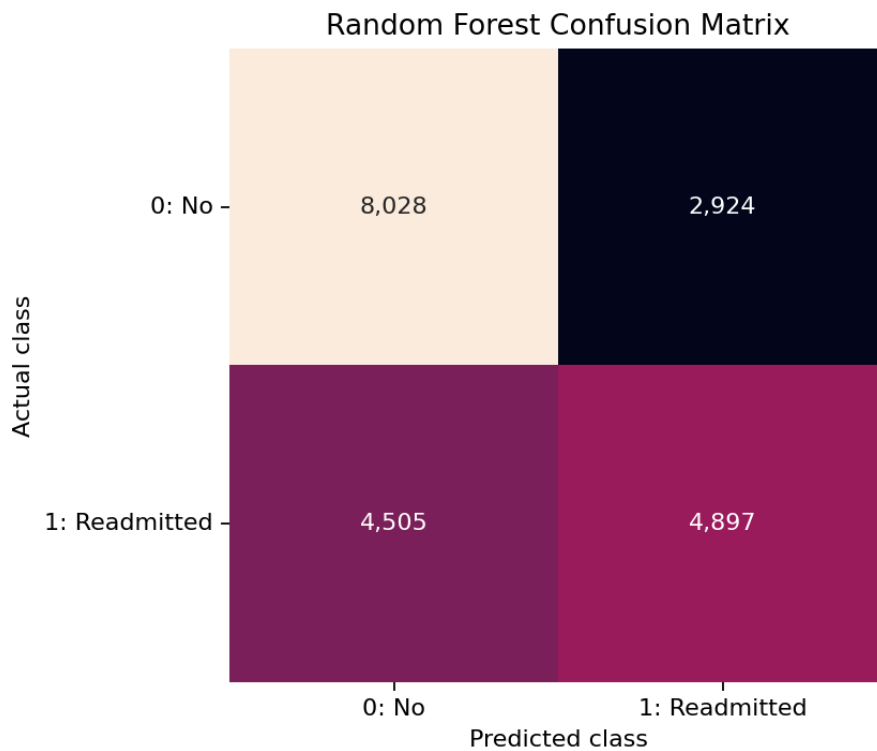


Figure 6. Confusion matrix on the held-out test set.

The model correctly classified 8,028 non-readmission encounters and 4,897 readmission encounters. It produced 2,924 false positives and 4,505 false negatives. The positive-class recall of 52.08% means that the model identified approximately half of the readmitted encounters in the held-out set. For a clinical screening system, the relative cost of false negatives and false positives would need to be defined before selecting a decision threshold.

9.2 Feature importance

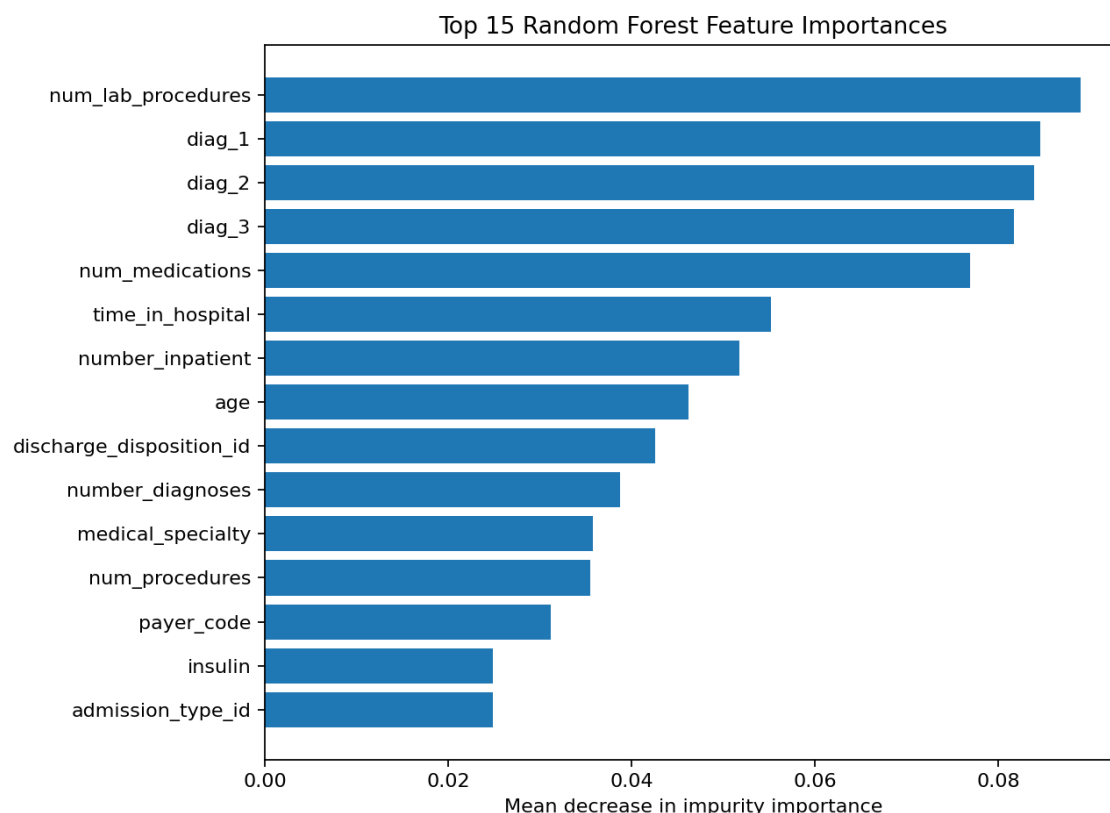


Figure 7. Fifteen highest Random Forest feature-importance values in the reproduced experiment.

The highest-ranked variables in the reproduced forest include the number of laboratory procedures, diagnosis fields, number of medications, time in hospital, prior inpatient utilization, age, discharge disposition, and number of diagnoses. These rankings describe how the fitted trees used variables for impurity reduction; they do not demonstrate that the variables cause readmission.

9.3 Why accuracy alone is insufficient

The model's 63.50% accuracy is not sufficient by itself to characterize usefulness. The positive-class recall is 52.08%, meaning that a substantial fraction of actual readmissions are missed in the reproduced test set. A deployment-oriented study would therefore need threshold analysis, precision-recall curves, calibration, cost-sensitive evaluation, subgroup analysis, and external validation.

10. Limitations, Risks, and Responsible Use

10.1 Data leakage risk

The current preprocessing is performed before the train-test split. Median/mode imputation and categorical encoding therefore have access to the full dataset. A stronger research design would fit all preprocessing transformations on the training partition only and apply the learned transformations to validation and test data.

10.2 Encoding limitations

Label encoding maps categories to integers. For tree models this can still be workable, but it introduces an arbitrary numeric ordering that is not intrinsically meaningful. One-hot encoding or an appropriate categorical treatment would be preferable for a stronger experimental pipeline.

10.3 Target simplification

Combining <30 and >30 into a single positive class removes clinically meaningful timing information. A future study could model the original three categories or formulate separate early-readmission and later-readmission tasks.

10.4 Single split and limited tuning

The study uses one 80/20 split and a fixed Random Forest configuration. No cross-validation, hyperparameter optimization, baseline comparison, or uncertainty interval is reported. The resulting performance should therefore be regarded as one reproducible benchmark rather than a definitive estimate.

10.5 Clinical validity

The model has not been prospectively validated, externally validated, calibrated for clinical use, tested for fairness, or evaluated for impact on patient outcomes. It must not be used as an autonomous clinical decision-maker.

10.6 Interactive risk score

The Prediction module's heuristic score is not the Random Forest output and has no demonstrated calibration or clinical validity. The paper therefore treats it as an educational user-interface feature only.

10.7 Privacy and governance

Any future deployment involving real patient data would require appropriate privacy, security, access control, governance, documentation, and clinical oversight. Public benchmark data should not be treated as evidence that a model is ready for use on a local hospital population.

11. Reproducibility and Deployment

11.1 Software stack

Component	Role
Python 3.x	Programming language
Pandas	Data loading and transformation
NumPy	Numerical operations
Matplotlib	Visualization
Seaborn	Statistical visualization
Scikit-learn	Modeling and evaluation
Streamlit	Interactive web application
Git / GitHub	Version control and repository hosting

11.2 Repository structure

```
Hospital-Readmission-Prediction/  
■■■ app.py  
■■■ README.md  
■■■ requirements.txt  
■■■ LICENSE  
■■■ data/  
■■■ notebooks/  
■■■ research/  
■■■ reports/  
■■■ screenshots/  
■■■ assets/  
■■■ docs/  
■■■ models/
```

11.3 Reproduction steps

- Place the dataset file `diabetic_data.csv` in the application directory.
- Install the required Python packages listed by the project.
- Run the Streamlit application with: `streamlit run app.py`
- Open the local Streamlit address shown by the application.
- Select the Machine Learning module to reproduce the Random Forest evaluation.
- For a publication-quality experiment, record the exact package versions, dataset checksum/version, preprocessing configuration, and environment.

11.4 Reproducibility record for this paper

Item	Recorded value
Dataset rows	101,766
Dataset columns	50
Model	RandomForestClassifier
<code>n_estimators</code>	100
<code>random_state</code>	42
<code>test_size</code>	0.20
Target mapping	NO=0; >30=1; <30=1
Identifier removal	<code>encounter_id</code> ; <code>patient_nbr</code>

Item	Recorded value
Imputation	numeric median; categorical mode
Categorical encoding	LabelEncoder per object column

12. Future Work

The project can be strengthened substantially without changing its overall educational purpose. The following improvements would turn the current prototype into a more rigorous experimental study.

- Use a leakage-safe preprocessing Pipeline and ColumnTransformer fitted only on training data.
- Compare one-hot encoding, ordinal treatment where justified, and native categorical approaches.
- Evaluate Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and calibrated baselines.
- Use stratified cross-validation and report confidence intervals or fold-level variability.
- Tune hyperparameters using a validation strategy that is separated from the final test set.
- Report ROC-AUC and especially precision-recall performance for the positive class.
- Evaluate calibration so that predicted probabilities can be interpreted quantitatively.
- Analyze performance across age, sex, race, and other clinically relevant subgroups.
- Preserve the original three-level target or evaluate separate early and later readmission outcomes.
- Replace the heuristic Prediction module with the actual trained model and a properly calibrated probability output.
- Add SHAP or another carefully validated explanation method for individual and global interpretation.
- Perform external validation on a distinct hospital population before making any clinical claims.
- Add model versioning, dataset checksums, automated tests, and a reproducible environment specification.
- Conduct prospective human-factors and clinical utility studies before considering real-world use.

12.1 Proposed stronger experimental design

A next-stage experiment should begin with a frozen test set. All preprocessing should be learned inside cross-validation on the training data. Multiple baseline models should be compared under the same split strategy, followed by hyperparameter tuning and calibration. The final selected model should be evaluated once on the untouched test set and then externally validated. This would make the performance claims considerably stronger than those supported by the current single-split prototype.

13. Conclusion

This study presents a complete healthcare analytics prototype for hospital readmission prediction using the Diabetes 130-US Hospitals dataset. The supplied implementation performs data cleaning, missing-value treatment, identifier removal, categorical encoding, exploratory analysis, Random Forest classification, evaluation, feature-importance visualization, and interactive Streamlit presentation.

Under the supplied configuration, the reproduced Random Forest achieved 63.50% accuracy on the held-out test set, with 62.61% precision, 52.08% recall, and 56.87% F1-score for the positive readmission class. These values provide a concrete baseline for the project and replace the placeholder metrics present in the earlier report.

The most important conclusion is methodological: the prototype demonstrates an end-to-end machine-learning workflow, but it does not establish clinical effectiveness. The current preprocessing introduces potential leakage, the categorical encoding is simplistic, the target combines early and later readmission, the model is evaluated on a single split, and the interactive Prediction page uses a heuristic score rather than the trained Random Forest. These limitations are material and should remain visible in any public presentation of the work.

With leakage-safe preprocessing, stronger validation, calibrated probabilities, subgroup analysis, external testing, and a prediction interface connected directly to the validated model, the project could serve as a stronger foundation for a formal research study.

Final status: suitable as a transparent educational/portfolio preprint draft with reproduced numerical results; not a clinically validated decision-support system.

References

- [1] B. Strack, J. P. DeShazo, C. Gennings, J. Olmo, S. Ventura, K. J. Cios, and J. N. Clore, “Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records,” *BioMed Research International*, vol. 2014, Article ID 781670, 2014.
- [2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems*, 2017.
- [5] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You? Explaining the Predictions of Any Classifier,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [6] J. Clore, K. Cios, J. DeShazo, and B. Strack, *Diabetes 130-US Hospitals for Years 1999–2008*, UCI Machine Learning Repository, Dataset 296, DOI: 10.24432/C5230J.
- [7] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [8] Python Software Foundation, *Python Documentation*.
- [9] Streamlit Inc., *Streamlit Documentation*.
- [10] NumPy Developers, *NumPy Documentation*.
- [11] Pandas Development Team, *pandas Documentation*.
- [12] Matplotlib Development Team, *Matplotlib Documentation*.
- [13] M. McKinney, *Python for Data Analysis*, 3rd ed., O’Reilly Media, 2022.

Appendix A. Dataset Variable Inventory

The supplied CSV contains 50 columns. The inventory below preserves the variable names from the dataset so that the paper remains directly traceable to the implementation.

No.	Variable	Type
1	encounter_id	int64
2	patient_nbr	int64
3	race	object
4	gender	object
5	age	object
6	weight	object
7	admission_type_id	int64
8	discharge_disposition_id	int64
9	admission_source_id	int64
10	time_in_hospital	int64
11	payer_code	object
12	medical_specialty	object
13	num_lab_procedures	int64
14	num_procedures	int64
15	num_medications	int64
16	number_outpatient	int64
17	number_emergency	int64
18	number_inpatient	int64
19	diag_1	object
20	diag_2	object
21	diag_3	object
22	number_diagnoses	int64
23	max_glu_serum	object
24	A1Cresult	object
25	metformin	object
26	repaglinide	object
27	nateglinide	object
28	chlorpropamide	object
29	glimepiride	object
30	acetohexamide	object
31	glipizide	object
32	glyburide	object
33	tolbutamide	object
34	pioglitazone	object
35	rosiglitazone	object
36	acarbose	object

No.	Variable	Type
37	miglitol	object
38	troglitazone	object
39	tolazamide	object
40	examide	object
41	citoglipton	object
42	insulin	object
43	glyburide-metformin	object
44	glipizide-metformin	object
45	glimepiride-pioglitazone	object
46	metformin-rosiglitazone	object
47	metformin-pioglitazone	object
48	change	object
49	diabetesMed	object
50	readmitted	object

Appendix B. Reproduced Classification Report

The following values are generated from the supplied dataset and the supplied model configuration.

Metric	Class 0	Class 1
Precision	0.640549	0.626135
Recall	0.733017	0.520847
F1-score	0.683670	0.568658
Support	10,952	9,402

Confusion matrix

	Predicted 0	Predicted 1
Actual 0	8,028	2,924
Actual 1	4,505	4,897

Top 20 feature importances

Rank	Feature	Importance
1	num_lab_procedures	0.089032
2	diag_1	0.084612
3	diag_2	0.083868
4	diag_3	0.081756
5	num_medications	0.076958
6	time_in_hospital	0.055203
7	number_inpatient	0.051769
8	age	0.046240
9	discharge_disposition_id	0.042605
10	number_diagnoses	0.038729
11	medical_specialty	0.035751
12	num_procedures	0.035518
13	payer_code	0.031159
14	insulin	0.024874
15	admission_type_id	0.024865
16	admission_source_id	0.021382
17	race	0.019753
18	gender	0.018119
19	number_outpatient	0.017691
20	number_emergency	0.014985

Appendix C. Key Implementation Details

C.1 Data loading and preprocessing

```
df = pd.read_csv("diabetic_data.csv")
data = df.copy()
data.replace("?", np.nan, inplace=True)

drop_cols = ["encounter_id", "patient_nbr"]
for col in drop_cols:
    if col in data.columns:
        data.drop(col, axis=1, inplace=True)

# Numeric: median imputation
# Categorical: mode imputation
# Target: NO=0, >30=1, <30=1
# Remaining object columns: LabelEncoder
```

C.2 Model training and evaluation

```
X = data.drop("readmitted", axis=1)
y = data["readmitted"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(
    y_test, y_pred, output_dict=True
)
```

C.3 Interpretation of the supplied Prediction module

The interactive Prediction page constructs a heuristic risk score using eight manually entered variables. Thresholds of 0.60 and 0.35 are used to display High, Moderate, or Low risk. Because the score is not learned from the Random Forest and has not been calibrated or validated, the number shown to the user should be treated only as an illustrative demonstration.

C.4 Recommended publication note

When sharing the project publicly, retain the distinction between (a) the reproduced Random Forest test-set experiment and (b) the heuristic dashboard risk interface. Do not describe the heuristic output as a validated model probability.

End of final preprint.